EasyLearning — API Test Case Document

Project: EasyLearning — CS Student Skills Tutoring Marketplace

Document version: 1.0

Documentation date: 13 October 2026 **Test plan status:** Planned cases have not been manually executed

Table of Contents

1. Testing Overview
2. GET and POST Endpoint Inventory
3. Detailed API Test Cases
4. Test Data
5. Execution Instructions
6. Execution Summary and Defect Log

1. Testing Overview

1.1 Purpose and scope

This document defines positive, negative, boundary, authentication, authorization, ownership, and dependency checks for API routes implemented with GET or POST. It is a test plan, not proof that the listed manual cases have been executed. Endpoint paths and role checks were derived from the Express mounts and route handlers.

1.2 Environment and tools

Item	Requirement
API base URL	Local default `http://localhost:5000`; use the actual API origin for a configured test environment
Database	PostgreSQL with the current application schema; session and payment table DDL must be obtained from the authoritative database owner because it is not included in repository schema initializers
Private storage	Supabase configuration for document/report attachment cases
Payment provider	PayFast sandbox and public callback configuration for end-to-end provider tests
Test tools	Existing backend tests use Node.js built-in `node:test`; manual calls can use curl or Postman
Postman collection	No collection was identified in the inspected repository

1.3 Preconditions

- Start the API with a current database connection and complete schema.
- Use separate fictional accounts with assigned role and approval state as applicable.
- For cookie-authenticated browser tests, retain the HttpOnly `token` cookie. Direct API tooling may authenticate with a test bearer token where supported.
- Do not share real student records, real credentials, private keys, or signed attachment URLs.
- Attachments use multipart form-data; application documents require both document fields, while reports accept one optional attachment.
- External-service tests are blocked until the corresponding private configuration and test service are available.

1.4 Request conventions

- JSON submissions use `Content-Type: application/json`.
- Browser requests use credentials/cookies; a bearer token may be used for server APIs where `authenticateToken` is present.
- Multipart uploads must not set a manual content-type boundary; let the HTTP client generate it.
- Each test case below defaults to **Not Run**. Record actual responses only after execution.

2. GET and POST Endpoint Inventory

Only implemented GET and POST routes are included in this inventory.

Module	Method	Endpoint	Authentication and permissions	Purpose
Health	GET	`/`	Public	Basic server-running response
Health	GET	`/api/test`	Public	Basic API response
Authentication	POST	`/api/auth/register`	Public	Register student account; tutor intent does not grant Tutor role
Authentication	POST	`/api/auth/login`	Public	Verify credentials and issue cookie
Authentication	GET	`/api/auth/me`	Authenticated	Retrieve current account and effective roles
Authentication	POST	`/api/auth/upgrade-tutor`	Authenticated	Legacy status check; does not grant Tutor access
Authentication	POST	`/api/auth/logout`	Public cookie-clearing endpoint	Clear the auth cookie
Tutor discovery	GET	`/api/tutors`	Public	List/search approved tutors
Tutor profile	GET	`/api/tutors/:id`	Public	Retrieve an approved tutor profile
Tutor profile	GET	`/api/tutors/profile`	Authenticated `TUTOR`	Retrieve caller's tutor profile
Tutor applications	GET	`/api/tutor-applications/subjects`	Authenticated	List available tutor subjects
Tutor applications	GET	`/api/tutor-applications/me`	Authenticated	Retrieve caller's application and safe document metadata
Tutor applications	POST	`/api/tutor-applications`	Authenticated `STUDENT`; private storage required	Submit an application and two documents
Tutor applications	GET	`/api/tutor-applications/:id/documents/:type`	Authenticated; applicant owner or `ADMIN`; storage required	Download authorized private document
Tutor applications	GET	`/api/tutor-applications`	Authenticated `ADMIN`	List submitted applications
Session requests	GET	`/api/session-requests/mine`	Authenticated `STUDENT`	List caller's session requests
Session requests	POST	`/api/session-requests`	Authenticated `STUDENT`	Create request to approved tutor
Session requests	GET	`/api/session-requests/incoming`	Authenticated `TUTOR`	List requests addressed to caller
Admin	GET	`/api/admin/dashboard`	Authenticated `ADMIN`	Retrieve admin overview, users, tutors, payments
Admin	GET	`/api/admin/payments`	Authenticated `ADMIN`	Retrieve payment records
Payments	POST	`/api/payments/initialize`	Authenticated `STUDENT`; own accepted session	Create/reuse pending PayFast checkout
Payments	GET	`/api/payments/session/:sessionRequestId`	Authenticated `STUDENT`; owner only	Retrieve session payment status
Payments	GET	`/api/payments/receipt/:sessionRequestId`	Authenticated `STUDENT`; owner only	Retrieve paid receipt
Payments	POST	`/api/payments/payfast/notify`	Provider callback; signature, IP and server validation	Process provider notification
Reports	POST	`/api/reports`	Authenticated `STUDENT`, `TUTOR`, or `ADMIN`	Submit report and optional private attachment
Reports	GET	`/api/reports/mine`	Authenticated supported role	Retrieve caller's reports only
Reports	GET	`/api/reports/admin`	Authenticated `ADMIN`	Retrieve/filter report queue
Reports	POST	`/api/reports/:id/status`	Authenticated `ADMIN`	Update report status and user-visible response
Reports	GET	`/api/reports/:id/attachment`	Authenticated report owner or `ADMIN`	Obtain authorized private attachment redirect

3. Detailed API Test Cases

All actual-result fields are intentionally blank and every case is marked **Not Run**. Replace these only after the particular request has been executed and its response recorded.

3.1 Health and authentication

Test Case ID	Module	Test Scenario	Preconditions	HTTP Method	Endpoint	Request Data	Expected Result	Actual Result	Status	Notes
HEALTH-001	Health	API process responds	API running	GET	`/`	None	`200`; JSON indicates server is running		Not Run	Does not prove database health
HEALTH-002	Health	API test endpoint responds	API running	GET	`/api/test`	None	`200`; JSON indicates API works		Not Run	
AUTH-001	Authentication	Register eligible student	Valid unused fictional student identity and university email	POST	`/api/auth/register`	JSON: `firstName`, `lastName`, `university`, `email`, `password`	`201`; safe user object with `STUDENT`; auth cookie set		Not Run	Never record real password
AUTH-002	Authentication	Register with tutor intent	Eligible unused identity	POST	`/api/auth/register`	Valid registration JSON and `role: "TUTOR"`	`201`; role remains `STUDENT`, not `TUTOR`		Not Run	Tutor requires application approval
AUTH-003	Authentication	Reject duplicate email	Existing account	POST	`/api/auth/register`	Valid fields using existing email	`400`; duplicate-account message; no new account		Not Run	
AUTH-004	Authentication	Reject invalid university/email pair	No account needed	POST	`/api/auth/register`	Valid JSON with mismatched or unsupported university/email	`400`; validation error		Not Run	Check university configuration
AUTH-005	Authentication	Reject public administrator registration	No account needed	POST	`/api/auth/register`	JSON with reserved admin email or admin role	`400`; no admin account created		Not Run	Admin provisioning is trusted server operation
AUTH-006	Authentication	Login with correct credentials	Existing fictional account	POST	`/api/auth/login`	JSON `email`, `password`	`200`; safe user and roles; HttpOnly token cookie		Not Run	
AUTH-007	Authentication	Reject incorrect credentials	Existing fictional account	POST	`/api/auth/login`	Correct email, incorrect password	`401`; generic invalid-credentials response		Not Run	
AUTH-008	Authentication	Reject missing login fields	API running	POST	`/api/auth/login`	Missing email or password	`400`; required-fields error		Not Run	
AUTH-009	Authentication	Report database DNS/connectivity failure safely	Test DB mock/proxy configured to return `ENOTFOUND` or network failure	POST	`/api/auth/login`	Valid-shaped login JSON	`503`; safe database-unavailable message; no hostname, connection string, or stack trace		Not Run	Automated test currently mocks nested DNS failure
AUTH-010	Authentication	Read current account with valid token	Valid student/tutor/admin token	GET	`/api/auth/me`	Cookie or bearer token	`200`; current profile and effective roles; no password hash		Not Run	
AUTH-011	Authentication	Reject missing or invalid token	No valid token	GET	`/api/auth/me`	No token or malformed/expired token	`401`; no private user data		Not Run	
AUTH-012	Authentication	Check legacy tutor-upgrade endpoint for unapproved student	Authenticated student without approved application	POST	`/api/auth/upgrade-tutor`	Empty JSON or no body	`409`; application URL; role remains student		Not Run	Endpoint must not bypass approval
AUTH-013	Authentication	Clear session cookie on logout	Any browser/API client	POST	`/api/auth/logout`	No body	`200`; token cookie cleared		Not Run	Route does not require token

3.2 Tutor discovery and applications

Test Case ID	Module	Test Scenario	Preconditions	HTTP Method	Endpoint	Request Data	Expected Result	Actual Result	Status	Notes
TUTOR-001	Tutor discovery	List approved tutors	API and database available	GET	`/api/tutors`	None	`200 { tutors: [...] }`; only approved tutors		Not Run	
TUTOR-002	Tutor discovery	Filter tutors by subject/university/search	Matching approved tutor exists	GET	`/api/tutors?search=...&subject=...&university=...`	URL-encoded optional filters	`200`; returned tutors match provided filters		Not Run	
TUTOR-003	Tutor profile	Retrieve approved tutor profile	Approved tutor ID known	GET	`/api/tutors/:id`	Replace `:id` with approved tutor ID	`200 { tutor: ... }`		Not Run	

Test Case ID	Module	Test Scenario	Preconditions	HTTP Method	Endpoint	Request Data	Expected Result	Actual Result	Status	Notes
TUTOR-004	Tutor profile	Reject invalid or absent tutor ID	API available	GET	`/api/tutors/:id`	Non-numeric ID, then valid-but-absent ID	Invalid ID: `400`; absent profile: `404`		Not Run	
TUTOR-005	Tutor profile	Tutor retrieves own profile	Authenticated approved tutor	GET	`/api/tutors/profile`	Tutor cookie/token	`200 { tutor: ... }`		Not Run	
TUTOR-006	Tutor profile	Student cannot retrieve tutor-only profile	Authenticated student	GET	`/api/tutors/profile`	Student cookie/token	`403`; no profile data		Not Run	
APP-001	Tutor application	Retrieve available subjects	Any authenticated account	GET	`/api/tutor-applications/subjects`	Valid auth cookie/token	`200 { subjects: [...] }`		Not Run	
APP-002	Tutor application	Applicant retrieves own absent application	Authenticated student without application	GET	`/api/tutor-applications/me`	Student token	`200`; `application: null`		Not Run	
APP-003	Tutor application	Applicant retrieves submitted application	Authenticated applicant with application	GET	`/api/tutor-applications/me`	Applicant token	`200`; own application and safe document statuses/metadata		Not Run	Ensure no storage key or signed URL leaks
APP-004	Tutor application	Submit valid application and both documents	Student, private storage configured, valid required data/files	POST	`/api/tutor-applications`	Multipart application fields, `registration` PDF/JPG/JPEG/PNG and `academicRecord` PDF/JPG/JPEG/PNG	`201`; application and both documents `PENDING`		Not Run	Each document max 5 MiB
APP-005	Tutor application	Reject application missing either document	Student and storage configured	POST	`/api/tutor-applications`	Multipart with only one required document	`400`; no submitted application		Not Run	
APP-006	Tutor application	Reject unsupported or oversized document	Student and storage configured	POST	`/api/tutor-applications`	Multipart with unsupported type or file over 5 MiB	`415` or upload-limit error; object not persisted		Not Run	Test each boundary separately
APP-007	Tutor application	Require authentication and student role for submission	Unauthenticated client, then tutor-only/admin client	POST	`/api/tutor-applications`	Multipart request	`401` unauthenticated; `403` disallowed role		Not Run	
APP-008	Tutor application	Administrator lists submitted applications	Authenticated admin and submitted applications exist	GET	`/api/tutor-applications`	Admin token	`200 { applications: [...] }` with safe document metadata		Not Run	
APP-009	Tutor application	Non-admin cannot list applications	Authenticated student or tutor	GET	`/api/tutor-applications`	Non-admin token	`403`; no application queue		Not Run	
DOC-001	Private documents	Applicant downloads own document	Applicant has document; storage configured	GET	`/api/tutor-applications/:id/documents/:type`	Applicant token and own application/document type	`302` to short-lived signed URL		Not Run	Do not save/share URL
DOC-002	Private documents	Admin downloads an applicant document	Admin; document exists; storage configured	GET	`/api/tutor-applications/:id/documents/:type`	Admin token	`302` to short-lived signed URL		Not Run	
DOC-003	Private documents	Other student cannot download document	Different authenticated student	GET	`/api/tutor-applications/:id/documents/:type`	Other user's token, target ID/type	`404`; no signed URL issued		Not Run	
DOC-004	Private documents	Handle unavailable private storage	Storage config absent or mocked failure	GET	`/api/tutor-applications/:id/documents/:type`	Authorized account	`503` if unconfigured; safe `500` on storage error		Not Run	External configuration required

3.3 Session requests and administration

Test Case ID	Module	Test Scenario	Preconditions	HTTP Method	Endpoint	Request Data	Expected Result	Actual Result	Status	Notes
SESSION-001	Session request	Student reads own requests	Authenticated student with requests	GET	`/api/session-requests/mine`	Student token	`200`; only records owned by caller		Not Run	
SESSION-002	Session request	Non-student cannot read student request list	Authenticated tutor/admin without student role	GET	`/api/session-requests/mine`	Token	`403`		Not Run	Dual-role STUDENT/TUTOR account may pass STUDENT guard
SESSION-003	Session request	Submit valid request to approved tutor	Authenticated student; approved tutor and profile exist	POST	`/api/session-requests`	JSON `tutorId`, `subject`, optional `requestedDate`, `message`	`201`; request status `PENDING`; student ID from token		Not Run	
SESSION-004	Session request	Reject missing/invalid request values	Authenticated student	POST	`/api/session-requests`	Missing subject, invalid tutor ID/date, oversized message	`400`; no request created		Not Run	Message limit 3,000 chars

Test Case ID	Module	Test Scenario	Preconditions	HTTP Method	Endpoint	Request Data	Expected Result	Actual Result	Status	Notes
SESSION-005	Session request	Reject self-request or unapproved tutor	Authenticated student	POST	`/api/session-requests`	Own ID or non-approved tutor ID	Self: `400`; tutor missing/unapproved: `404`		Not Run	
SESSION-006	Session request	Tutor reads only incoming requests	Authenticated tutor with requests for multiple tutors	GET	`/api/session-requests/incoming`	Tutor token	`200`; only requests where target tutor is caller		Not Run	
ADMIN-001	Admin	Retrieve admin dashboard	Authenticated admin	GET	`/api/admin/dashboard`	Admin token	`200`; overview, user records, approved tutors, payment records		Not Run	Requires database tables used by dashboard
ADMIN-002	Admin	Non-admin is denied dashboard	Authenticated student or tutor	GET	`/api/admin/dashboard`	Non-admin token	`403`; no admin data		Not Run	
ADMIN-003	Admin	Admin retrieves payment queue	Authenticated admin and payment tables available	GET	`/api/admin/payments`	Admin token	`200 { payments: [...] }`		Not Run	Payment schema is not included in repository DDL

3.4 Payments

Test Case ID	Module	Test Scenario	Preconditions	HTTP Method	Endpoint	Request Data	Expected Result	Actual Result	Status	Notes
PAY-001	Payments	Initialize checkout for owned accepted request	Student owns accepted request; valid rate; PayFast configured	POST	`/api/payments/initialize`	JSON `{ "sessionRequestId": 123 }`	`200`; PayFast URL and server-generated checkout fields/signature		Not Run	External sandbox integration needed
PAY-002	Payments	Reject invalid or unsupported initialization body	Authenticated student	POST	`/api/payments/initialize`	Missing/invalid ID or extra client-supplied amount	`400`; no payment initialized		Not Run	Amount is server-derived
PAY-003	Payments	Reject request not owned by caller	Authenticated student; another student's request	POST	`/api/payments/initialize`	Other user's ID	`404`; no ownership disclosure		Not Run	
PAY-004	Payments	Reject pending or declined request	Authenticated student owns request not accepted	POST	`/api/payments/initialize`	Owned request ID	`409`; payment not initialized		Not Run	Only accepted sessions are payable
PAY-005	Payments	Report missing PayFast configuration	PayFast settings absent	POST	`/api/payments/initialize`	Valid owned accepted request	`503`; configuration unavailable error		Not Run	
PAY-006	Payments	Retrieve own payment status	Authenticated student owns session	GET	`/api/payments/session/:sessionRequestId`	Valid owned ID	`200`; status, amount/currency and paid time		Not Run	
PAY-007	Payments	Reject invalid or foreign session status lookup	Authenticated student	GET	`/api/payments/session/:sessionRequestId`	Invalid ID, then another user's ID	`400` invalid; `404` not owned/not found		Not Run	
PAY-008	Payments	Retrieve receipt for paid session	Authenticated owner; payment is verified paid	GET	`/api/payments/receipt/:sessionRequestId`	Owned session ID	`200 { receipt: ... }`		Not Run	
PAY-009	Payments	Do not expose unpaid/foreign receipt	Authenticated student	GET	`/api/payments/receipt/:sessionRequestId`	Unpaid or another user's session	`404`; no receipt data		Not Run	
PAY-010	Payments	Reject invalid PayFast notification	Callback test harness/provider simulation	POST	`/api/payments/payfast/notify`	URL-encoded notification with invalid signature, source, amount, or details	Relevant `400` / `403`; payment not marked paid		Not Run	Must not test against live merchant accidentally
PAY-011	Payments	Record valid notification once	PayFast sandbox callback and current pending payment	POST	`/api/payments/payfast/notify`	Valid URL-encoded notification	`200`; payment transition recorded once after provider validation		Not Run	External IP/server validation needed
PAY-012	Payments	Handle repeated valid notification safely	Previously recorded valid notification	POST	`/api/payments/payfast/notify`	Same valid provider notification	`200` already recorded; no duplicate receipt transition		Not Run	

3.5 Reports

Test Case ID	Module	Test Scenario	Preconditions	HTTP Method	Endpoint	Request Data	Expected Result	Actual Result	Status	Notes
REPORT-001	Reports	Submit valid report	Authenticated supported role	POST	`/api/reports`	Multipart category, subject, description	`201`; `OPEN`; reporter ID/roles derived from auth		Not Run	
REPORT-002	Reports	Reject missing or invalid fields	Authenticated account	POST	`/api/reports`	Missing subject/description or unknown category	`400`; no report created		Not Run	

Test Case ID	Module	Test Scenario	Preconditions	HTTP Method	Endpoint	Request Data	Expected Result	Actual Result	Status	Notes
REPORT-003	Reports	Reject text over limits or spoofed fields	Authenticated account	POST	`/api/reports`	Subject >160, description >5,000, reporter ID/status extras	`400`; no identity/status override		Not Run	
REPORT-004	Reports	Reject unauthenticated submission	No token	POST	`/api/reports`	Valid multipart fields	`401`; no report created		Not Run	
REPORT-005	Reports	Validate optional private attachment	Authenticated account; storage configured	POST	`/api/reports`	Valid report and valid PDF/JPG/JPEG/PNG ≤5 MiB	`201`; safe metadata only; object remains private		Not Run	Test signature mismatch and oversize separately
REPORT-006	Reports	Retrieve own report history	Authenticated reporter with reports	GET	`/api/reports/mine`	Reporter token	`200`; own reports only; user-visible response only		Not Run	
REPORT-007	Reports	Prevent cross-user report/attachment access	Different authenticated user	GET	`/api/reports/mine`, then `/api/reports/:id/attachment`	Other user's report ID	History excludes foreign report; attachment returns `404`; no signed URL		Not Run	
REPORT-008	Reports	Admin retrieves and filters queue	Authenticated admin and reports exist	GET	`/api/reports/admin?status=OPEN&category=Technical%20issue`	Admin token	`200`; matching records with full description and reporter		Not Run	
REPORT-009	Reports	Non-admin cannot retrieve report queue	Authenticated student/tutor	GET	`/api/reports/admin`	Non-admin token	`403`; no queue data		Not Run	
REPORT-010	Reports	Admin updates status and response	Authenticated admin; report exists	POST	`/api/reports/:id/status`	JSON status and optional response	`200`; persisted status/response appears in owner's history		Not Run	
REPORT-011	Reports	Reject invalid report ID/status	Authenticated admin	POST	`/api/reports/:id/status`	Malformed ID or unsupported status	`400`; no change		Not Run	
REPORT-012	Reports	Handle database/storage failure safely	Test database/storage failure injected	POST	`/api/reports`	Valid report; optional upload	`500` / `503`; no secrets in response; orphan upload cleaned up when possible		Not Run	Local route tests mock services

4. Test Data

Use fictional identities and placeholders. Never use production accounts or include literal passwords/tokens in this document.

Account	Example identifier	Setup
Student	`student.one@example.test`	Register through the supported flow using a test university identity accepted by local validation, or seed through an approved test fixture
Approved Tutor	`tutor.approved@example.test`	Create student, submit application/documents, then use admin workflow to verify both documents and approve
Pending Tutor Applicant	`tutor.pending@example.test`	Register student and submit application; leave review pending
Rejected Tutor Applicant	`tutor.rejected@example.test`	Use test data with a rejected application or rejected document
Administrator	`admin@universitydomain` (source default; configurable)	Provision only via trusted `npm run admin:bootstrap` with a private `ADMIN_PASSWORD`

Use `<TEST_PASSWORD>` and `<TEST_TOKEN>` placeholders in execution tools. Public registration does not provision `ADMIN`. For stable automated tests, use the repository's existing mock-based tests rather than modifying a shared production database.

5. Execution Instructions

5.1 Start the API

Install dependencies and configure the server privately, then:

```
cd server
npm start
```

The startup schema initializers must complete before the API listens. Ensure the connected database also has the session-request and payment tables expected by the API; their DDL is not supplied in the repository.

5.2 Obtain authentication

1. Register a permitted test student or use an already provisioned test account.
2. Send a login request to `/api/auth/login`.
3. Preserve the returned HttpOnly cookie in a browser/Postman cookie jar, or use the test token mechanism supported by the backend for direct bearer-token calls.
4. Do not paste tokens, cookies, or passwords into reports or source control.
5. Admin credentials must come from a trusted administrator bootstrap process; do not create an admin using public registration.

5.3 Send JSON requests

Example student login:

```
curl -i -X POST "$API_BASE_URL/api/auth/login" \
  -H "Content-Type: application/json" \
  -c cookies.txt \
  -d '{"email":"student.one@example.test","password":"<TEST_PASSWORD>"}'
```

Example authenticated personal report history:

```
curl -i "$API_BASE_URL/api/reports/mine" \
  -b cookies.txt
```

Example submit a session request:

```
curl -i -X POST "$API_BASE_URL/api/session-requests" \
  -b cookies.txt \
  -H "Content-Type: application/json" \
  -d '{"tutorId":123,"subject":"Data Structures","message":"Please help me prepare."}'
```

5.4 Submit multipart requests

For tutor applications, send application fields plus one `registration` file and one `academicRecord` file. For reports, send `category`, `subject`, `description`, and optionally `attachment`. Use only allowed file formats and test-sized fictional material. Allow curl/Postman to generate multipart boundaries.

5.5 Automated tests and external dependencies

Run the existing server suite:

```
cd server
npm test
```

The client `npm test` script invokes Node's test runner, but no client test suite was identified. Supabase and PayFast test coverage in the repository mocks external operations; live provider/SMTP testing requires explicit test configuration and must not be performed against real users or payments without authorization.

6. Execution Summary and Defect Log

6.1 Summary

Measure	Result
Planned manual API cases	67

Measure	Result
Manual cases passed	0 recorded
Manual cases failed	0 recorded
Manual cases blocked	0 recorded
Manual cases not run	67
Automated server suite run during documentation preparation	69 passed, 0 failed

The 69 passing automated cases are repository test-runner results, not execution of the 67 manual test cases above. Many use mocked PostgreSQL, storage, or payment services and do not establish production-service availability.

6.2 Defect log

Defect ID	Related test case	Description	Severity	Reproduction steps	Status